CYPRESS PARA LEIGOS

Colinha do curso · tudo o que você precisa ter à mão na hora de escrever um teste

COLINHA · 1/2

1 TERMINAL: OS COMANDOS

`node -v`	confere se o Node está instalado
`npm init -y`	transforma a pasta num projeto (cria o package.json)
`npm install cypress --save-dev`	instala o Cypress no projeto
`npx cypress open`	abre a janela do Cypress (pra escrever e ver)
`npx cypress run`	roda tudo sem janela, com resumo no final
`npm test`	atalho pro cypress run (aula 16)

2 ONDE FICA CADA COISA

```
meu-primeiro-teste/
├─ cypress/
│  ├─ e2e/          ← os seus testes
│  │  ├─ login.cy.js
│  │  └─ compra.cy.js
│  └─ support/
├─ node_modules/    ← não mexa
├─ cypress.config.js
├─ package.json     ← a ficha do projeto
└─ .gitignore
```

Todo arquivo de teste termina em `.cy.js`, é assim que o Cypress sabe que é teste.

3 A RECEITA DE UM TESTE

```
describe('Login', () => {          // o caderno de receitas
  beforeEach(() => {               // roda antes de CADA it
    cy.visit('https://www.saucedemo.com')
  })

  it('entra com usuário e senha', () => {  // uma receita
    cy.get('[data-test="username"]').type('standard_user')
    cy.get('[data-test="password"]').type('secret_sauce')
    cy.get('[data-test="login-button"]').click()
    cy.contains('Products').should('be.visible')  // a conferência
  })
})
```

Todo teste é assim:

1. **abre** o site (visit)
2. **acha** o elemento (get)
3. **faz** alguma coisa (type, click)
4. **confere** o resultado (should)

Sem o `should`, o teste só clica: ele não confere nada.

4 COMANDOS DO CYPRESS

`cy.visit(url)`	abre o site
`cy.get(seletor)`	acha um elemento na página
`.type('texto')`	digita num campo
`.click()`	clica
`cy.contains('texto')`	acha pelo texto que aparece
`.select('valor')`	escolhe opção de um menu
`.first()`	pega só o primeiro da lista
`.should(...)`	confere (a asserção)

5 AS CONFERÊNCIAS (SHOULD)

`'be.visible'`	está aparecendo na tela
`'contain', 'txt'`	tem esse texto no meio
`'have.text', 'txt'`	o texto é exatamente esse
`'not.exist'`	não existe na página
`'have.value', 'x'`	o campo tem esse valor
`'be.disabled'`	o botão está desativado

O Cypress espera sozinho (até 4 segundos) a conferência dar certo. Não precisa de espera.

Site de treino: saucedemo.com · usuário **standard_user** · senha **secret_sauce**

CYPRESS PARA LEIGOS

Seletores, erros, Git e as palavras que você vai ouvir no trabalho

COLINHA · 2/2

6 QUAL SELETOR USAR? (AULA 13)

1º MELHOR **data-test**

`[data-test="username"]`

feito pra testes: não muda quando o visual muda

2º **id**

`#user-name`

o "CPF" do elemento: não se repete na página

3º SE PRECISAR **classe ou texto**

`.btn` `cy.contains('Login')`

muda fácil, o teste quebra à toa

Como achar: botão direito no elemento › **Inspecionar**. Ou, no Cypress, clique na **mira** (Selector Playground), passe o mouse no elemento e copie o seletor que ele sugere.

7 O TESTE FICOU VERMELHO? FAÇA 3 PERGUNTAS (AULA 15)

1
Onde parou?
o passo vermelho na lista da esquerda

2
O que ele esperava?
a mensagem em vermelho: "expected..."

3
O que tinha na tela?
clique no passo e olhe o site naquele momento

`Timed out retrying`	não achou o elemento: confira o seletor (letra errada? aspas?)
`expected ... to contain`	achou, mas o texto é outro: confira a mensagem do site
`npx não é reconhecido`	o Node não está instalado ou o VS Code precisa ser reaberto (aula 29)
`scripts desabilitados`	no PowerShell: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`

8 GIT E GITHUB (AULA 19)

`git init`	liga o Git na pasta (só 1 vez)
`git add .`	separa todos os arquivos
`git commit -m "msg"`	salva o jogo, com um nome
`git push`	manda pro GitHub

Antes do primeiro commit, crie o `.gitignore` com `node_modules` dentro.

9 ATALHOS DO VS CODE

`Ctrl + S`	salvar o arquivo
`Ctrl + '`	abrir e fechar o terminal
`Ctrl + Z`	desfazer
`Ctrl + /`	comentar a linha
`Ctrl + P`	achar um arquivo pelo nome

10 GLOSSÁRIO: AS PALAVRAS DO DIA A DIA

QA: quem cuida da qualidade do sistema, testando antes de chegar no cliente

Teste automatizado: um código faz os passos e confere sozinho

Spec: um arquivo de teste (o seu `login.cy.js`)

Seletor: o "endereço" de um elemento na página

Teste negativo: testar o que acontece quando o usuário erra

Bug: o sistema fazendo algo diferente do esperado

Repositório: a pasta do projeto guardada no GitHub

Teste manual: uma pessoa usa o sistema e confere tudo na mão

E2E: de ponta a ponta, usa o sistema como um usuário de verdade

Asserção: a conferência do teste, o `should`

Runner: a janela do Cypress que mostra os passos e o site

Regressão: rodar tudo de novo pra ver se nada quebrou

CI: testes rodando sozinhos sempre que o código muda

Commit: um "save" do projeto, com nome e data